

UNIVERSIDADE FEDERAL DO RIO GRANDE DO NORTE
CENTRO DE CIÊNCIAS EXATAS E DA TERRA
DEPARTAMENTO DE MATEMÁTICA APLICADA
DIM0129 - ORGANIZAÇÃO DE COMPUTADORES

OLIVE OLIVEIRA MEDEIROS
YASMIN GIORDANO SANTOS

RELATÓRIO DO PROJETO:

Implementação de um modelo de organização para uma arquitetura de conjunto de instruções compatível com a filosofia RISC, executando “pipeline”

NATAL/RN

2026

1. INTRODUÇÃO

O presente relatório tem como objetivo cumprir requisito avaliativo da disciplina DIM0129 - Organização de Computadores, ministrada pelo Prof. Dr. Márcio Eduardo Kreutz, em conjunto com o código em SystemC do projeto disponibilizado em <https://github.com/minkyzecapagods/pipeline-systemc> e vídeo explicativo disponibilizado em [inserir link do vídeo].

1.1 Resumo

O presente projeto detalha a concepção e implementação de um modelo de organização de um microprocessador em SystemC. A arquitetura do sistema é dividida fundamentalmente em duas partes integradas: a Parte Operativa (PO), responsável pelo fluxo e transformação dos dados, e a Parte de Controle (PC), responsável por orquestrar os sinais que guiam a execução.

O processador foi desenhado sob a filosofia RISC (Reduced Instruction-Set Computer). Seu grande diferencial é a execução baseada em um "Pipeline" de 5 estágios (Busca, Decodificação, Execução, Memória e Write-Back), simulando o funcionamento de arquiteturas clássicas como o MIPS. O sistema lida ativamente com a decodificação de instruções em linguagem de máquina (hexadecimal), calculando endereços, resolvendo desvios condicionais e escrevendo os resultados de volta no banco de registradores de forma síncrona aos ciclos de clock. O processador foi desenvolvido e validado em um ambiente de simulação digital utilizando a biblioteca SystemC e visualização de ondas no GTKWave.

1.2 Objetivos do Projeto

A implementação do microprocessador foi realizada com o objetivo de simular um sistema digital capaz de executar instruções de forma paralelizada através de um pipeline de 5 estágios. O projeto almeja integrar a Parte de Controle e a Parte Operativa para processar um conjunto específico de doze instruções.

Nesse sentido, o sistema foi elaborado para garantir a execução das seguintes operações: lógicas (AND, OR, XOR, NOT), aritméticas e de comparação (ADD, SUB, CMP), de acesso à memória (LD, ST) e de saltos de fluxo (J, JN, JZ). Além disso, buscou-se tratar as dependências inerentes à execução em pipeline (dados e controle) e adequar o sistema para simulações baseadas na leitura de arquivos de texto com algoritmos pré-compilados.

2. ARQUITETURA DO SISTEMA

A apresentação do desenho do projeto será dividida em conformidade com as decisões arquiteturais essenciais exigidas para a construção da máquina. Para um olhar mais detalhado dos módulos do sistema, como ULA, Extensor de Sinal e Multiplexadores, deve-se verificar o código em SystemC supramencionado.

2.1 Decisões de Projeto

A apresentação do desenho do projeto será dividida em conformidade com as decisões arquiteturais essenciais exigidas para a construção da máquina. Para um olhar mais detalhado dos módulos do sistema, como ULA, Extensor de Sinal e Multiplexadores, deve-se verificar o código em SystemC supramencionado.

Para que a arquitetura RISC fosse viabilizada no código, as seguintes especificações e decisões de projeto foram adotadas:

- **Tamanho da palavra do processador:** A arquitetura foi fixada em 32 bits, sendo este o tamanho padrão dos dados trafegados entre os componentes da Parte Operativa.
- **Formato da palavra de instrução:** O formato da instrução segue o modelo de 32 bits, dividido em campos como Opcode (6 bits iniciais), identificadores de registradores de origem e destino (5 bits cada) e valores imediatos/funct na parcela final da palavra.
- **Modos de endereçamento de operandos:** Foram implementados o endereçamento por registrador (para operações aritméticas/lógicas), endereçamento imediato (para carregamento de constantes, usando a adaptação da instrução LRI) e o endereçamento base-deslocamento (para o acesso às memórias com LD e ST).
- **Tamanho do banco de registradores:** O processador conta com um banco de 32 registradores, cada um armazenando palavras de 32 bits.
- **Tamanho das memórias de instruções e de dados:** Ambas as memórias foram projetadas como arrays de 32 bits, abstraídas na simulação com capacidade para comportar o endereçamento dos algoritmos de teste.
- **Número e tipos de barramentos:** O sistema utiliza múltiplos fios (sinais SystemC) paralelos de 32 bits dedicados para interligar os estágios da Parte Operativa, além de barramentos de controle de 1, 2, 4 ou 5 bits para seleção de MUXes e controle da ULA.

- **Organização do pipeline:** O pipeline foi rigorosamente implementado com 5 estágios distintos: Instruction Fetch (IF), Instruction Decode (ID), Execution (EX), Memory (MEM) e Write-Back (WB).

2.1 Diagrama de Blocos da Organização (PO + PC)

A organização do sistema baseia-se em registradores de barreira (IF/ID, ID/EX, EX/MEM, MEM/WB) que isolam cada um dos 5 estágios.

Na Parte Operativa (PO), o fluxo inicia no Program Counter (PC) e na Memória de Instruções. O dado flui para o Banco de Registradores, passa pelo Extensor de Sinal e chega à Unidade Lógica Aritmética (ULA) ou ao Somador de Branch. Dali, o resultado pode acessar a Memória de Dados e, finalmente, retorna ao Banco de Registradores através do multiplexador de Write-Back. A Parte de Controle (PC) está centralizada no Estágio de Decodificação (ID), que lê o Opcode e emite os sinais de controle (RegDst, AluSrc, MemRead, etc.) que viajam paralelamente aos dados pelas barreiras de pipeline até seus destinos finais.

Diagrama 1 - Funcionamento Geral do Sistema

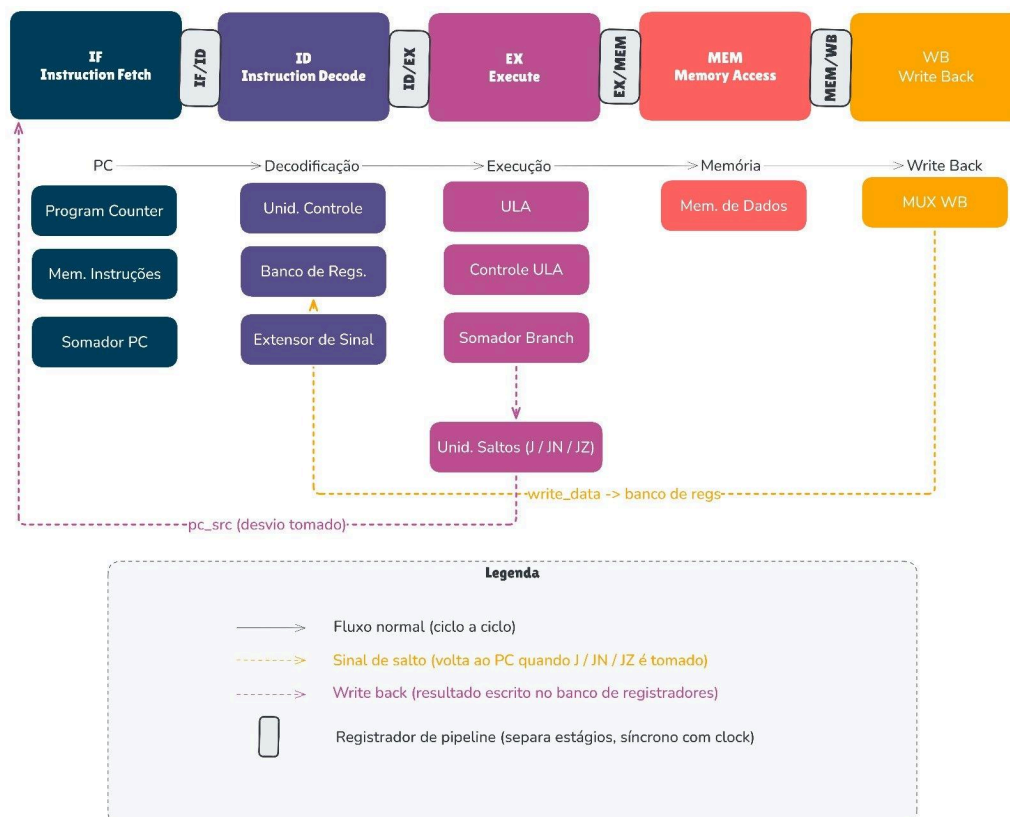
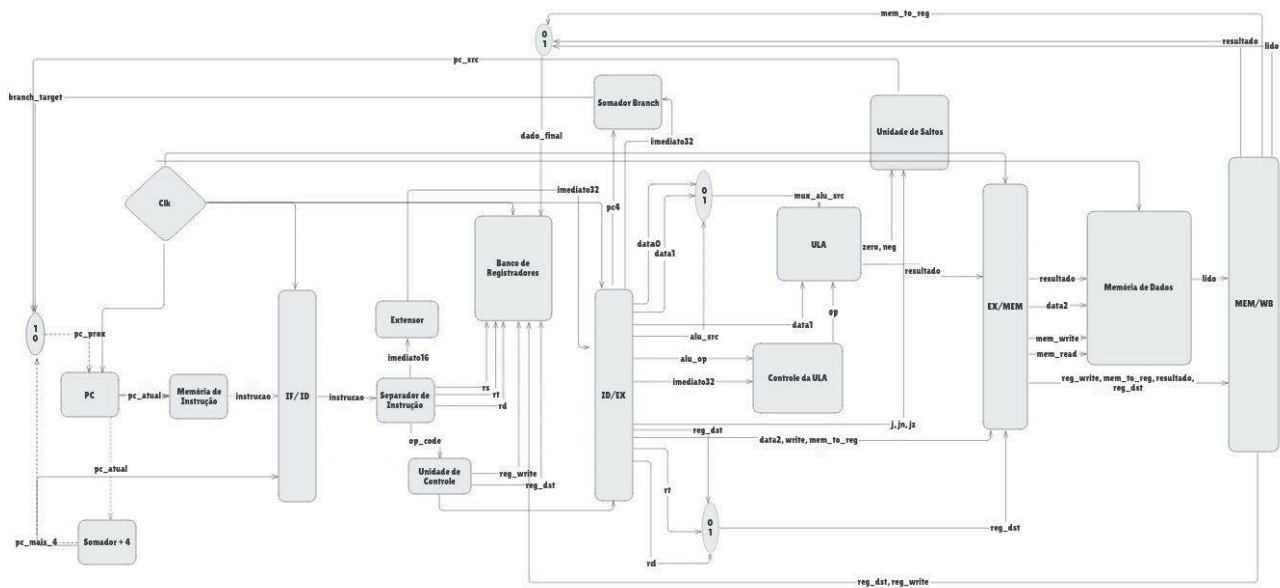


Diagrama 2 - Funcionamento Geral do Sistema



Os diagramas acima ilustram a comunicação orgânica entre a PO e a PC. O fluxo inicia no estágio de Busca (IF), onde o *Program Counter* endereça a Memória de Instruções. No estágio de Decodificação (ID), a instrução é separada: os operandos ativam a leitura no Banco de Registradores, enquanto o Opcode é lido pela Unidade de Controle, que "acende" os sinais corretos para aquela instrução específica.

Esses sinais e dados não viajam de uma vez. Eles são segurados pelas barreiras (Reg_ID_EX, por exemplo) e liberados ciclo a ciclo. No estágio de Execução (EX), a ULA realiza a matemática definida pelo controle. No estágio de Memória (MEM), o dado pode ser salvo ou lido da RAM. Por fim, no estágio de Write-Back (WB), o fio final dá a volta por fora do processador e conecta-se à entrada de escrita do Banco de Registradores, retroalimentando o sistema com os resultados processados.

3. COMPORTAMENTOS DO PIPELINE: TRATAMENTOS DE DEPENDÊNCIAS

O tópico a seguir dedica-se a descrever a lógica utilizada para tratar os comportamentos anômalos inerentes a qualquer arquitetura em pipeline. A análise em curso dividirá os problemas enfrentados em duas categorias principais: dependência de dados e dependência de controle.

3.1 Dependência de Dados e a Solução via Software

Durante a operação paralela, uma instrução de soma (ADD) pode necessitar de um dado que acabou de ser calculado por uma instrução anterior (LRI), mas que ainda não foi gravado no banco de registradores, visto que a gravação só ocorre no estágio final (WB). Para solucionar esse comportamento de dependência de dados, optou-se pela estratégia de inserção de bolhas (Instruções NOP - 0x00000000) no código. Assim, o algoritmo aguarda ciclos de clock ociosos suficientes para que a instrução anterior conclua seu estágio de Write-Back, garantindo que o dado esteja fisicamente disponível para leitura no estágio de Decodificação (ID) da instrução dependente.

3.2 Dependência de Controle e o "Branch Delay Slot"

O processador implementa instruções de saltos (J, JN, JZ). Como a Unidade de Saltos toma a decisão de desviar o fluxo do Program Counter apenas no estágio de Execução (EX), as duas instruções subsequentes ao salto já entraram no pipeline (estão nos estágios IF e ID). Para resolver esse conflito sem a necessidade de descartar o processamento (Flush), adotou-se o comportamento de Branch Delay Slot. Essa técnica consiste em deixar com que o processador termine a execução dessas duas instruções subsequentes normalmente, exigindo que o programador coloque instruções úteis ou NOPs logo após os Jumps. Esse comportamento maximiza o desempenho, evitando o desperdício de ciclos de clock nas tomadas de decisão.

4. SIMULAÇÃO DO PROCESSADOR E ANÁLISE DE DESEMPENHO

Nesta seção, decorrerá a descrição de como foram realizados os testes práticos do microprocessador em SystemC. Serão explorados três algoritmos de teste obrigatórios que demonstram o funcionamento de toda a arquitetura.

4.1 Ambiente de Simulação

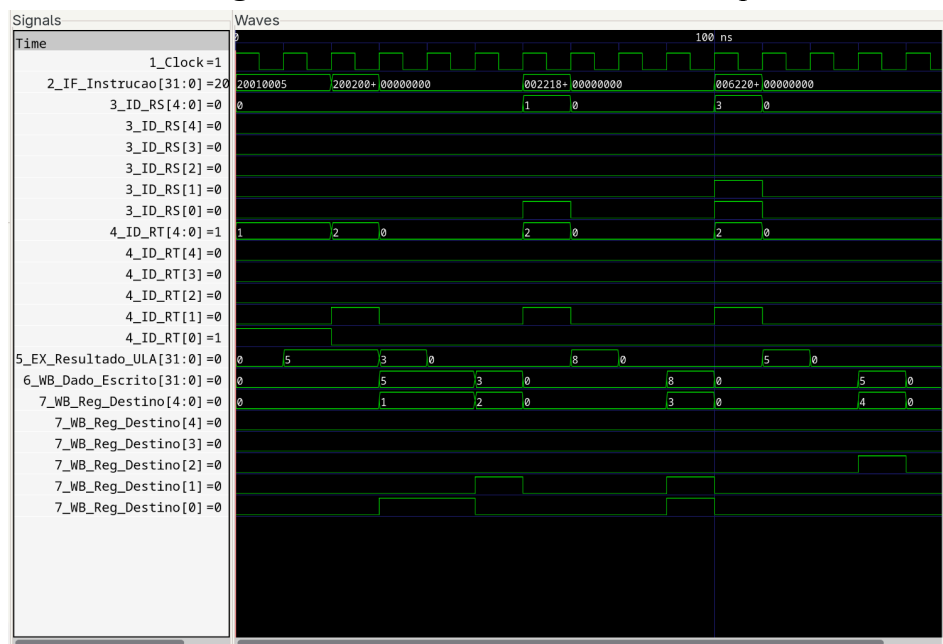
Dada a natureza do projeto, a fabricação do ambiente abstrato de simulações foi feita através da compilação do código SystemC utilizando o compilador G++. A leitura dos algoritmos foi automatizada: o arquivo main.cpp recebe um arquivo .txt contendo as instruções em hexadecimal e carrega esses dados na memória ROM antes de iniciar o clock.

A análise dos ciclos de relógio e do trajeto das informações pelos estágios do pipeline foi feita visualmente através da geração de arquivos .vcd abertos no visualizador GTKWave.

4.2 Casos Práticos de Teste Implementados (Três Algoritmos Escolhidos)

O primeiro caso trata de um algoritmo que carrega os valores 5 e 3 em registradores distintos e, após a inserção de bolhas (NOPs), executa as instruções de soma (ADD), somando o 3 com o 5 e subtração (SUB), subtraindo 3 de 8.

Imagem 1 - Recorte de tela do GTKWave após Teste 1

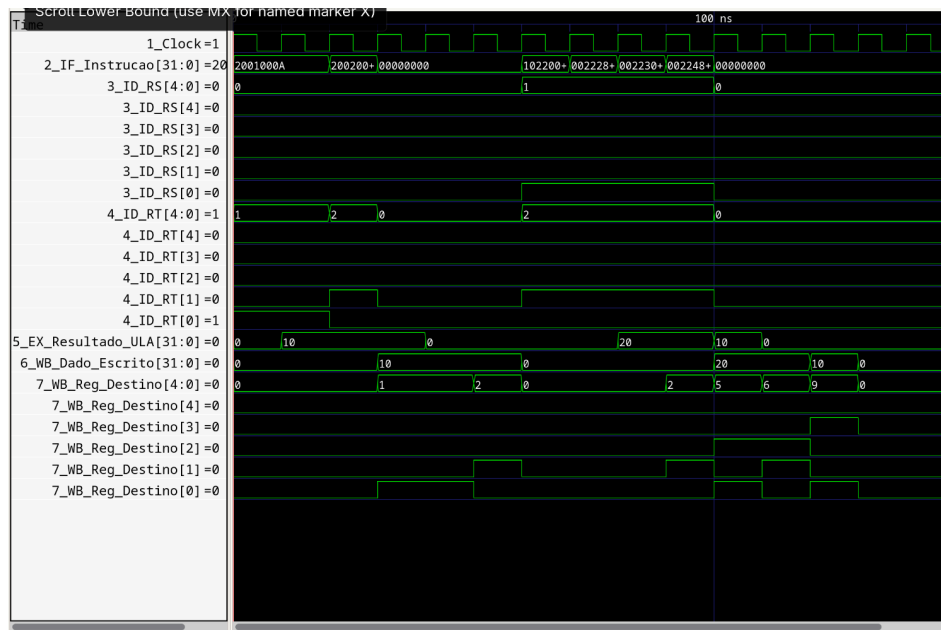


Na simulação de onda, foi possível observar claramente o atraso de 1 ciclo em cada barreira (Reg_ID_EX, Reg_EX_MEM) formando o clássico "efeito escadinha", culminando no salvamento dos resultados 5, 3, 8 e 5 nos registradores corretos durante o Write-Back.

O segundo caso de teste testa as instruções LD e ST. Um valor constante 15 é salvo em um endereço específico da RAM também calculada pela ULA a partir de um valor 4 também inserido no teste e em seguida a instrução LD busca o valor nesse mesmo endereço. A última instrução é a soma do valor 15 registrado na memória com ele mesmo.

Imagem 2 - Recorte de tela do GTKWave após Teste 2

Imagem 3 - Recorte de tela do GTKWave após Teste 3



O resultado acima demonstra o comportamento de desvio do processador (Instrução JZ). Na onda apresentada, o salto é acionado após uma comparação entre dois valores 10 gerar a flag de zero na ULA. A prova de que o tratamento do pipeline foi eficaz é observada no Write-Back: os resultados das duas instruções diretamente após o salto (Delay Slot) são salvos nos registradores, mas o Program Counter muda abruptamente seu fluxo, pulando um bloco inteiro de código e aterrissando no endereço alvo após verificar a igualdade. As instruções ignoradas jamais atingem a linha do WB_Reg_Destino.

4.2 Análise de Desempenho (Ciclos de Relógio)

A análise de desempenho em função do comportamento do pipeline demonstra uma melhoria massiva em relação a um processador monociclo. Devido ao paralelismo dos 5 estágios, após o ciclo 5 (quando o pipeline se enche pela primeira vez), o processador passa a concluir o Write-Back de uma instrução a cada novo ciclo de clock, tendendo a um Throughput de 1 instrução por ciclo (CPI = 1). A degradação dessa eficiência ocorre, apenas e intencionalmente, nos tratamentos implementados para as dependências de dados, onde os NOPs exigem o consumo de 3 ciclos ociosos para garantir a atualização segura do Banco de Registradores.

5. CONCLUSÃO E ANÁLISE DE RESULTADOS

Por fim, cumpre analisar os resultados aferidos, esmiuçando-se os problemas enfrentados ao longo da implementação em C++, de forma que os obstáculos também sejam levados em consideração em conjunto com a vitória final do projeto.

5.1 Problemas Encontrados na Implementação

Em síntese, adiante serão analisados os problemas enfrentados durante a escrita do código em SystemC.

5.1.1 Tipagem e Portas Desconectadas (*Unbound Ports*)

O rigor da tipagem de SystemC causou erros arquiteturais no código (como o erro fatal F4 - iface != 0). A tentativa de conectar sinais com sinal (sc_int) em portas sem sinal (sc_uint) resultava na finalização prematura da simulação. Isso foi resolvido através da criação de componentes dedicados, como o *SomadorBranch*, projetados especificamente para lidar com tipos assinalados, mantendo a harmonia do circuito sem uso de "casts" forçados em C++.

5.1.2 Conversão Hexadecimal vs Binária (*Opcodes*)

A transição do modelo mental de binário para os arquivos de teste em hexadecimal gerou problemas de tradução. Um exemplo foi o Opcode da instrução LRI (assumida como 8). Em notação hexadecimal (0x08), o compilador lia os bits equivocados e a Unidade de Controle não ativava as portas corretas. Esse obstáculo foi sanado refazendo cuidadosamente o mapeamento dos códigos nos arquivos de texto de teste, respeitando os 6 bits primários da palavra arquitetural.

5.2 Conclusão Geral

Todavia, apesar das limitações impostas pela linguagem e pela depuração gráfica baseada em ondas, a integração das máquinas e do pipeline obteve sucesso. As análises atestam que todas as doze instruções solicitadas cumprem suas rotas lógicas através da PO e da PC com pipeline, comprovando que a adoção do paralelismo e de técnicas clássicas (como o Branch Delay Slot) resultam na construção de um hardware funcional, escalável e fiel aos pilares da teoria de Organização de Computadores.

ANEXO I - INSTRUÇÕES PARA REPRODUZIR AS SIMULAÇÕES (RETIRADO DO *README.MD*)

Sistema do Microprocessador MIPS Pipeline (SystemC)

Este projeto implementa um processador RISC utilizando SystemC e um design baseado em módulos hierárquicos e registradores de barreira.

Compilação e Testes

Para compilar todos os arquivos e instanciar o executável da simulação, deve-se usar o comando `g++` apontando para os diretórios corretos da biblioteca SystemC.

Bash

```
# Comando de compilação geral
g++ -std=c++17 -I./include -I/usr/local/systemc-3.0.2/include -L/usr/local/systemc-3.0.2/lib-linux64
src/main.cpp -lsystemc -Wl,-rpath=/usr/local/systemc-3.0.2/lib-linux64 -o mips_sim
```

Executando os Algoritmos

O executável exige um argumento indicando o caminho do arquivo de teste em texto:

■ Teste de Lógica e Aritmética

```
./mips_sim tests/test1.txt
```

Teste de Memória

```
./mips_sim tests/test2.txt
```

Teste de Saltos e Dependência de Controle

```
./mips_sim tests/test3.txt
```

Após a execução, o arquivo `simulacao_mips_final.vcd` será gerado. É recomendável abrir este arquivo em ferramentas como o GTKWave para a visualização gráfica e temporal de todos os barramentos da Parte Operativa e de Controle.